



Reactive Forms

Persistent Interactive | Persistent University



Key Learning Points

1. Model driven (Reactive) forms
2. Creating Form Controls
3. Creating Form Group
4. Validation Properties
5. Reacting to Form Input changes

Reactive Forms

- To work with this approach, we need to import the **ReactiveFormsModule** which belongs to the **@angular/forms** package.
- In Reactive forms, form control objects are created in the component class.

```
import { FormControl } from '@angular/forms';  
export class SignupFormComponent {  
    name = new FormControl();  
    phone = new FormControl();  
}
```

- It is possible to set initial values and validations in the form control objects.

```
import { Validators } from '@angular/forms';  
name = new FormControl('Tina', Validators.required);  
phone = new FormControl('', Validators.minLength(8));
```

Group form control object using FormGroup

- Better approach is to group all form controls in one FormGroup.

```
export class SignupFormComponent
{
    signupForm: FormGroup;
}
```

- Add the form controls to the group.

```
export class SignupFormComponent {
    this.signupForm = new FormGroup ({
        name: new FormControl('Tina',Validators.required);
        phone: new FormControl('',Validators.minLength(8));
    })
}
```

Bind the FormGroup to the template form

- Bind the form using FormGroup.
- Bind the FormControl object to the input element as well.

```
<form class="form-horizontal"  
[FormGroup]="signupForm" >  
  <label>Name</label>  
  <input type="text" formControlName = "name" >  
</form>
```

Form value and validity status

- Use the json pipe to check the form value and validity status

<p>Form value {{ signupForm.value | json }}

</p>

<p> Form status {{ signupForm.status | json }}

</p>

Nested FormGroup

- When there are related form controls, it is possible to again group them using FormGroup.
- Eg: Password and ConfirmPassword.

```
this.signupForm = new FormGroup ({  
    name: new FormControl('Tina',Validators.required);  
    phone: new FormControl('',Validators.minLength(8));  
    password: new FormGroup({  
        pwd: new FormControl(),  
        confirmPwd: new FormControl()  
    }  
}),  
}
```

Binding to the template

Use of formControlName to bind.

```
<div formGroupName = "password">
  <div class="form-group">
    <label for="inputPassword">Password</label>
    <input type="password" formControlName = "pwd" id="inputPassword" ></div>
  <div class="form-group">
    <label for="confirmPassword" >Confirm Password</label>
    <input type="password" formControlName = "confirmPwd" id="confirmPassword" ></div>
</div>
```


Validations to implement

- All form control elements are required.
- Disable the submit button until the status of the form is VALID.
- The email field should strictly contain an email id.
- The password field should have a minimum length of 8.

Refactor using FormBuilder

- FormBuilder provides a compact way of defining the form control objects and the form group.
- Inject the FormBuilder in the component's constructor.

```
export class SignupFormComponent {  
  signupForm: FormGroup;  
  constructor(private fb: FormBuilder) {}  
}
```

```
ngOnInit() {  
  this.signupForm = this.fb.group({  
    name: ['', Validators.required],  
    phone: ['', Validators.minLength(8)]  
  })  
}
```

Input Field Validation Properties

Property	Description
\$pristine	The field has not been modified yet
\$dirty	The field has been modified
\$valid	The field content is valid
\$invalid	The field content is not valid
\$touched	The field has been touched
\$untouched	The field has not been touched

Form Validation Properties

Property	Description
\$pristine	No fields have not been modified yet
\$dirty	One or more fields have been modified
\$valid	The form content is valid
\$invalid	The form content is not valid
\$submitted	The form is submitted

Disable the submit button

- The submit button can be disabled with below expression.

```
<button type="submit" [disabled] = "!signupForm.valid" >Submit</button>
```

Validation Errors

- Check the validity using the properties `pristine`, `valid`, `invalid` and `dirty`, `touched` and `untouched`.

```
<div *ngIf="signupForm.controls.name.invalid &&  
  signupForm.controls.name.touched"  
  Name is invalid  
</div>
```

- The expression in the if condition is too long and can be shortened by using getters for the form controls.

Add getters for the form controls

- Add Getters in the component class for all the form control objects.

```
export class SignupFormComponent implements OnInit {  
    get name() { return this.signupForm.get('name'); }  
    get phone() { return this.signupForm.get('phone'); }  
}
```

- The template expression is shortened as below:

```
<div *ngIf="name.invalid && name.touched"  
    Name is invalid  
</div>
```

Making errors more explicit

- The invalid property is a Boolean value and is set to true when any one validation criteria fails.
- To find out explicit errors there is an error object for every form control.
- Eg: To display separate messages when phone is required and when the minimum length criteria fails.

```
<div *ngIf = "phone.errors?.required">  
  Phone field can't be blank  
</div>
```

```
<div *ngIf = "phone.errors?.minLength">  
  Phone has to be more than 8 characters  
</div>
```


Submit the form

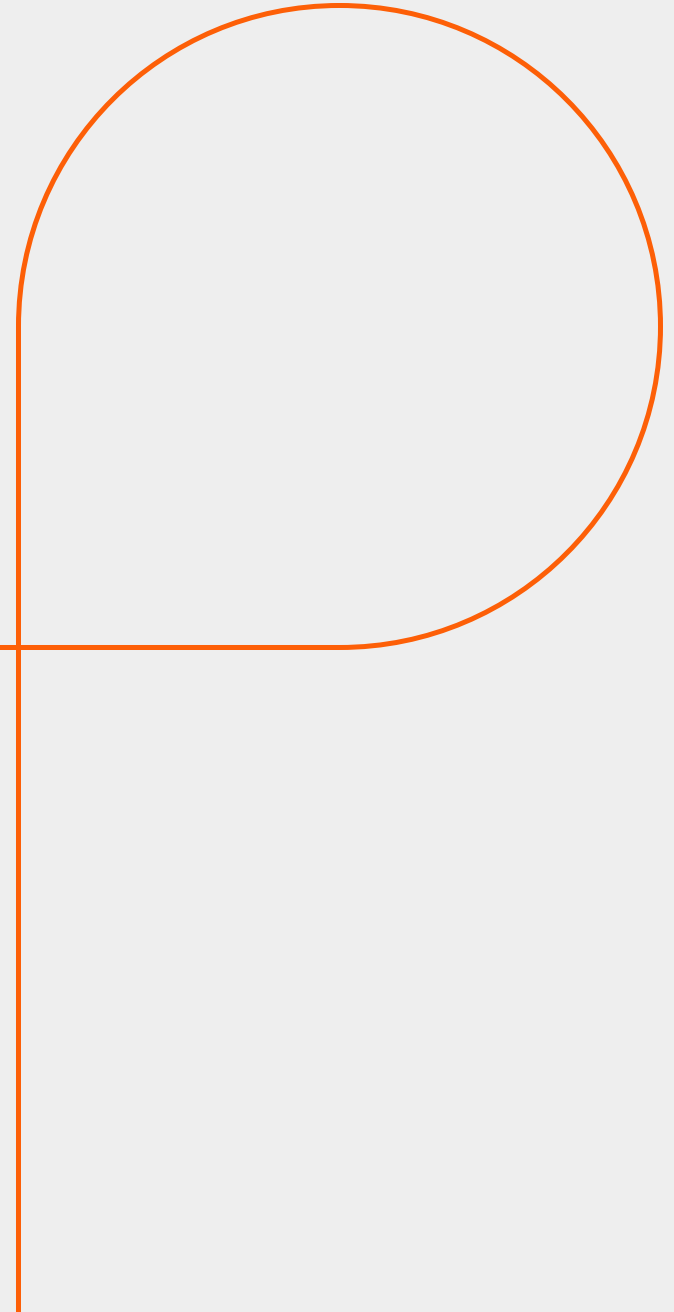
- The form can be submitted using `ngSubmit`.

```
<form class="form-horizontal"  
[formGroup]="signupForm"  
  (ngSubmit)="onFormSubmit()" >
```

- Access the form values in the component using `signupForm` object.

```
public onFormSubmit() {  
  if(this.signupForm.valid) {  
    this.user = this.signupForm.value;  
    console.log(this.user);  
    /* HTTP Post request*/  
  }  
}
```

Reacting to Form changes

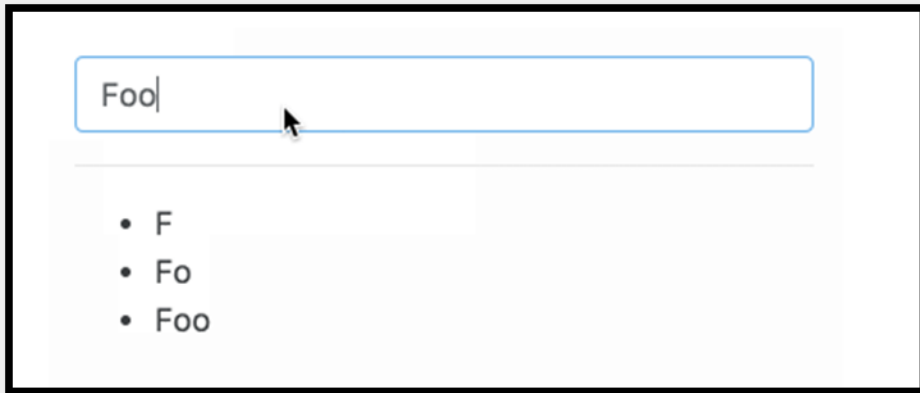


React to Form changes

- We can process a model driven form the traditional way by calling a function on submission of the form and then processing the form action there.
- But with Angular we also have another option of processing the form by means of an observable chain which we then subscribe to.
- Reactive forms are better when there needs to be some real-time processing of the form as the user types in content.
- Depending on the requirement and need, any of the approaches can be used.

Search Field Example

- Both FormControl and FormGroup expose an observable called valuesChanged.
- By subscribing to this observable we can react in real-time to changing values of an individual form control, or a group of form controls.
- Eg: Search field implementation where as the user types keywords, call an API to fetch the words that start with the keyword.



Implementation

- In the component, define a form control object for the search field and array for searches.

```
class ReactiveFormComponent implements OnInit {  
  searchField: FormControl;  
  searches: string[] = [];  
}
```

- Create a form control object for the search field

```
ngOnInit() {  
  this.searchField = new FormControl();  
}
```

Implementation

- Add an input field to the template and bind it to the control object.

```
<input type="search" class="formcontrol"  
[formControl]="searchField" >
```

- Display the list of searches in a list.

```
<ul>  
  <li *ngFor="let search of searches">{{ search }}</li>  
</ul>
```

Subscribing to form changes

- Since the form control object 'searchField' exposes an observable 'valueChanges', we can subscribe to it.

```
ngOnInit() {  
  this.searchField = new FormControl();  
  this.searchField.valueChanges  
    .subscribe(term => {  
    this.searches.push(term);  
    //Ideally make an API call here  
  });  
}
```

Every new search keyword entered by user, is being pushed to the array

- Ideally for every keyword typed, the list of words must be fetched from an API.

debounceTime operator

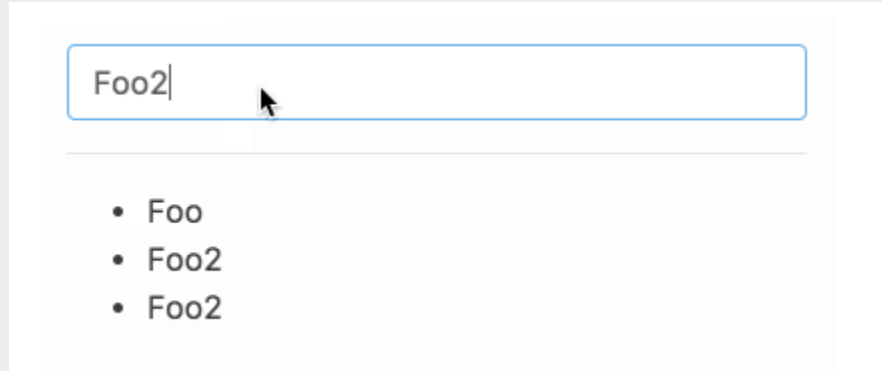
- If for every keypress, we make an API call, it will lead to too many HTTP requests.
- An efficient solution is to make an API call only when user stops typing.
- An operator from RxJS **debounceTime** helps to achieve this.

```
this.searchField.valueChanges
  .debounceTime(400)
  .subscribe(term => {
    this.searches.push(term);
    //Ideally make an API call here
  });
```

- If user has stopped typing for 400ms, only then the new value will be published.

Further optimization

- Possibility of same search keyword being entered twice by user.



A screenshot of a search input field. The input field contains the text "Foo2". Below the input field, a dropdown menu is visible, showing three suggestions: "Foo", "Foo2", and "Foo2". A mouse cursor is positioned over the input field.

- This would trigger a second unnecessary API request.
- Ideally we only want to make the API call if the search term has changed.

distinctUntilChanged operator

- We can use called **distinctUntilChanged** operator from RxJS which only publishes to its output stream if the value being published is different from before.

```
this.searchField.valueChanges
  .debounceTime(400)
  .distinctUntilChanged()
  .subscribe(term => {
    this.searches.push(term);
  });
```

- This avoids making a duplicate Ajax call for same search key.

Template driven vs Reactive Forms

Template Driven

- Most of the code in the templates
- Uses ngModel for validation properties
- Validations are added directly in the templates.
- Need to import FormsModule
- Custom validation is possible only by creating a new directive

Reactive Forms

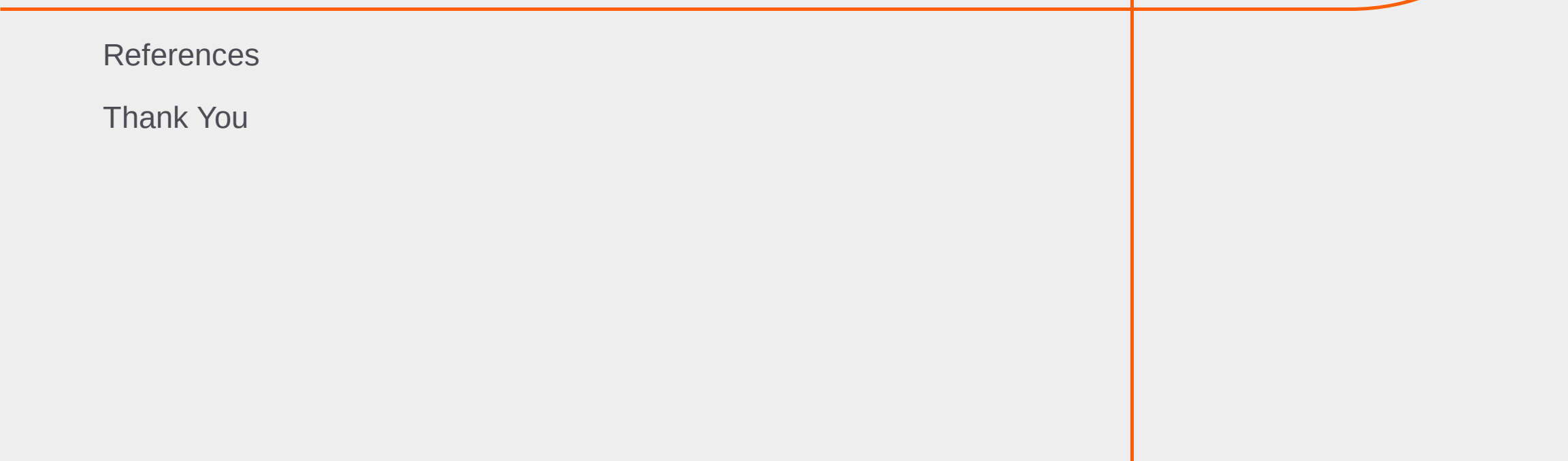
- Most of the code in the component class
- Uses the FormControl and FormGroup objects for validation properties.
- Validations are added in the FormControl objects.
- Need to import ReactiveFormsModule
- Custom validation is easily possible by writing a function and binding it to list of validators.

Summary: Session

With this we have come to an end of our session, where we discussed about

- How to work with ReactiveFormsModule
- Creating Form Controls and binding to inputs
- Validation properties
- Displaying appropriate error messages
- Reacting to input changes and calling appropriate functions

Appendix



References

Thank You

Key Contacts :

Persistent University :

- Archana Ghatigar
archana_ghatigar@persistent.co.in



Thank you!

Persistent Interactive | Persistent University

